

Attention contextuelle Write–Read–Value : étude d’un chemin architectural

Anonymous authors

Paper under double-blind review

Abstract

Nous étudions une attention contextuelle Write–Read–Value (W/R/V) pour un modèle de langage causal d’environ 98,2 millions de paramètres avec objets lexicaux résiduels liés et micro-experts déterministes. W/R/V emploie huit têtes de lecture contextuelles, deux têtes partagées d’écriture/valeur, une normalisation RMS par tête, un encodage positionnel rotatif et un résidu lexical initialisé à zéro dans chaque flux d’écriture. PyTorch SDPA est configuré pour autoriser le dispatch FlashAttention du softmax causal. Un entraînement terminé à 100M tokens atteint une loss d’évaluation de 4,706118 à 118 416 tokens/s d’entraînement journalisés sur un H100. Un entraînement GQA archivé atteint 4,820476 à 111 355 tokens/s, mais avec un learning rate maximal inférieur (3×10^{-4} contre 4×10^{-4}) et sans configuration réalisée colocalisée ; nous rapportons donc l’écart sans revendiquer une victoire architecturale alignée. Les ablations WVR à état fixe restent des contrôles négatifs informatifs, avec un meilleur résultat de 4,895427. Les preuves soutiennent la configuration W/R/V réalisée complète comme candidate à une réplification contrôlée, non un effet lexical résiduel isolé.

1 Introduction

Les modèles de langage fondés sur l’attention combinent transport contextuel et sélection dépendante du contenu parmi les représentations individuelles des tokens passés. Nous étudions si cette sélection peut être paramétrée comme une attention contextuelle Write–Read–Value (W/R/V), tandis que l’identité lexicale n’intervient que par un résidu étroit et lié.

Notre conception suit un principe de chemin partagé complété par des résidus. Chaque token traverse le même calcul dense. L’identité du token module des branches étroites d’objets lexicaux et de micro-experts ; un résidu lexical d’écriture initialisé à zéro est lié à la même table de rang 16. Le mélangeur principal est une attention causale W/R/V à huit têtes de lecture contextuelles et deux têtes contextuelles partagées d’écriture/valeur. Une normalisation RMS par tête et RoPE précèdent PyTorch SDPA.

Cette architecture provient d’une étude systématique de WVR à état fixe. Ces variantes sont compatibles avec une limitation d’une matrice comprimée unique pour la sélection d’occurrences. Nous les conservons comme ablations de chemin négatives plutôt que comme architecture finale.

Nos contributions sont empiriques :

- une attention contextuelle W/R/V avec têtes de lecture/écriture groupées, normalisation RMS par tête, RoPE et SDPA configuré pour autoriser FlashAttention ;
- un résidu lexical lié et initialisé à zéro sur les flux d’écriture, avec objets lexicaux feed-forward liés et micro-experts déterministes ;
- un résultat W/R/V terminé à 100M tokens de 4,706118 à 118 416 tokens/s ;
- des ablations de chemin qui motivent, sans les isoler, des hypothèses sur l’adressage comprimé ou lexical ;
- un rapport fondé sur les artefacts bruts qui sépare observations terminées et revendications architecturales appariées en optimisation.

Le run W/R/V observé est numériquement meilleur et plus rapide que notre run GQA archivé, mais les learning rates maximaux diffèrent. Nous ne revendiquons donc pas encore une victoire appariée en optimisation.

La confirmation requise est un nouveau GQA à 4×10^{-4} dans le même harness, ainsi que des checkpoints finaux sauvegardés et des ablations directes du résidu lexical.

2 Méthode

2.1 Attention contextuelle W/R/V

Soit $\mathbf{H} \in \mathbb{R}^{B \times T \times d}$ le tenseur des états cachés normalisés. Chaque couche projette des lectures, écritures et valeurs contextuelles,

$$\mathbf{R} = \mathbf{H}\mathbf{W}_r, \quad \mathbf{W}_{\text{ctx}} = \mathbf{H}\mathbf{W}_w, \quad \mathbf{V} = \mathbf{H}\mathbf{W}_v. \quad (1)$$

Le modèle évalué utilise huit têtes de lecture et deux têtes partagées d’écriture/valeur, de dimension 48. Un code déterministe du token et une projection apprise de la table lexicale liée de rang 16 forment une adresse lexicale $\mathbf{L}(x_t)$. Pour la tête d’écriture h ,

$$\mathbf{w}_{t,h} = \mathbf{w}_{t,h}^{\text{ctx}} + \tanh(g_h) \mathbf{L}_h(x_t), \quad g_h = 0 \text{ à l'initialisation.} \quad (2)$$

La branche lexicale est donc exactement neutre au départ, tandis que le flux contextuel reste actif. Lectures et écritures reçoivent des normalisations RMS indépendantes par tête et un encodage rotatif. Les têtes W/V sont répétées par groupes de quatre têtes R, puis l’attention causale est

$$\mathbf{y}_{t,h} = \sum_{j \leq t} \text{softmax}_j \left(\frac{\text{RoPE}(\mathbf{r}_{t,h})^\top \text{RoPE}(\mathbf{w}_{j,\kappa(h)})}{\sqrt{48}} \right) \mathbf{v}_{j,\kappa(h)}. \quad (3)$$

PyTorch SDPA est configuré pour autoriser le dispatch FlashAttention sur H100. W/R/V est donc bien un mécanisme d’attention ; sa terminologie décrit le flux et le résidu lexical, non une absence de softmax. En décodage incrémental, les tenseurs W et V passés sont conservés et croissent avec le préfixe.

2.2 Objet lexical résiduel partagé

La transformation feed-forward partagée est un SwiGLU dense,

$$\mathbf{F}(\mathbf{h}) = \mathbf{W}_d [\text{SiLU}(\mathbf{W}_g \mathbf{h}) \odot \mathbf{W}_u \mathbf{h}]. \quad (4)$$

Pour l’identité de token x_t , une table partagée $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times r}$ fournit une modulation de faible rang :

$$\mathbf{z}_t = \text{SiLU}(\mathbf{U}\mathbf{h}_t) \odot (\mathbf{1} + \mathbf{E}[x_t]), \quad (5)$$

$$\mathbf{O}(\mathbf{h}_t, x_t) = \mathbf{D}\mathbf{z}_t. \quad (6)$$

La même table $\mathbf{E}$ est partagée entre les couches ; les matrices de projection restent propres à chaque couche. Mettre $\mathbf{E}$ à zéro neutralise initialement la modulation dépendante du token, mais la branche objet reste active via une porte non nulle α :

$$\mathbf{F}_{\text{objet}}(\mathbf{h}_t, x_t) = \mathbf{F}(\mathbf{h}_t) + \alpha \mathbf{O}(\mathbf{h}_t, x_t). \quad (7)$$

2.3 Résidu de micro-expert déterministe

Chaque couche affecte le token x_t à l’un des N micro-experts selon

$$e_\ell(x_t) = (x_t + \ell) \bmod N. \quad (8)$$

Toutes les activations cachées expertes sont calculées dans une même opération tensorielle régulière, puis la sortie retenue est sélectionnée sans tri ni dispersion. Pour une largeur experte w ,

$$\mathbf{R}_\ell(\mathbf{h}_t, x_t) = \beta \mathbf{W}_d^{(e_\ell(x_t))} [\text{SiLU}(\mathbf{W}_g^{(e_\ell(x_t))} \mathbf{h}_t) \odot \mathbf{W}_u^{(e_\ell(x_t))} \mathbf{h}_t]. \quad (9)$$

La sortie feed-forward complète est $\mathbf{F} + \alpha \mathbf{O} + \beta \mathbf{R}$. Il s’agit d’un résidu lexical ajouté à un chemin dense partagé, et non d’une affirmation selon laquelle le routage remplacerait le calcul commun.

2.4 Ablation associative WVR à état fixe

L’ablation antérieure à état fixe utilise une matrice associative récurrente plutôt qu’une attention softmax. Nous appelons ce mécanisme *WVR* : adresse d’écriture (Write), valeur contextualisée (Value) et adresse de lecture (Read). Pour le token x_t et l’état caché $\mathbf{h}_t$, une forge lexicale compressée réutilise la table d’objets lexicaux liée,

$$\mathbf{f}_t = \mathbf{W}_{f,2} \text{SiLU}(\mathbf{W}_{f,1} \mathbf{E}[x_t]), \quad (10)$$

avec un goulot implémenté $16 \rightarrow 4 \rightarrow 128$. L’adresse commune combine ce code lexical déterministe avec une signature projetée de l’état caché,

$$\mathbf{c}_t = \text{norm}(\mathbf{W}_c \mathbf{h}_t), \quad (11)$$

$$\mathbf{a}_t = \text{norm}(\text{forge}(x_t) + \tanh(\eta) \mathbf{c}_t). \quad (12)$$

Le code lexical est une adresse liée à l’identité du token ; il ne constitue pas une preuve de représentation sémantique. La projection contextuelle est elle aussi décrite opérationnellement, sans revendication sémantique.

L’écriture emploie l’adresse précédente et la valeur cachée contextualisée courante,

$$\mathbf{w}_t = \mathbf{a}_{t-1}, \quad \mathbf{v}_t = \mathbf{W}_v \mathbf{h}_t, \quad \mathbf{r}_t = \mathbf{a}_t, \quad (13)$$

$$\hat{\mathbf{v}}_t = \mathbf{w}_t^\top \mathbf{M}_{t-1}, \quad (14)$$

$$\mathbf{M}_t = \lambda \mathbf{M}_{t-1} + \rho \mathbf{w}_t (\mathbf{v}_t - \hat{\mathbf{v}}_t)^\top, \quad (15)$$

$$\mathbf{y}_t = \mathbf{r}_t^\top \mathbf{M}_{t-1}, \quad (16)$$

$$\mathbf{c}_t^{\text{WVR}} = \gamma \mathbf{W}_o \mathbf{y}_t. \quad (17)$$

Le résidu de contexte $\mathbf{c}_t^{\text{WVR}}$ est ajouté à la sortie de la convolution causale. L’implémentation décale causalement le flux d’écriture : la lecture courante utilise $\mathbf{r}_t$, tandis que l’adresse d’écriture précédente est appariée au payload courant. La résolution triangulaire par segments constitue une parallélisation exacte de cette récurrence. WVR mémorise donc une valeur cachée contextualisée, et non un code lexical. L’état comprend une matrice de rang 128 ainsi que des vecteurs d’adresse et de charge de taille fixe, indépendamment de la longueur du préfixe.

La normalisation des collisions maintient une estimation diagonale amortie de charge $\mathbf{d}_t$ et redimensionne les coordonnées de lecture et d’écriture par $(\mathbf{d}_t + \epsilon)^{-1/2}$ avant normalisation. Elle ne partitionne pas la matrice de rang 128 en plusieurs mémoires.

L’attention QKV compare une requête contextualisée à chaque clé conservée et réalise une compétition normalisée entre occurrences individuelles. WVR lit au contraire l’association récurrente pré-mise-à-jour par $\mathbf{r}_t^\top \mathbf{M}_{t-1}$. Notre comparaison porte sur la qualité et le débit, sans crédit lié à la taille d’un KV cache. WVR possède un état fixe, mais des occurrences répétées ou en collision peuvent s’écraser.

L’extension expérimentale d’adresse d’occurrence remplace la projection contextuelle dense par une signature compacte $d \rightarrow 8 \rightarrow 128$ et ajoute une signature positionnelle déterministe contrôlée,

$$\mathbf{a}_t = \text{norm}(\text{forge}(x_t) + \text{smallextx}(\mathbf{h}_t) + \tanh(\gamma) \text{pos}(t)), \quad (18)$$

avec $\gamma = 0$ à l’initialisation et un unique compteur de position de taille fixe pour le décodage incrémental. Son entraînement terminé au budget complet est rapporté comme ablation négative.

2.5 Ablations du chemin architectural

Nous conservons trois conditions intermédiaires terminées : une couche W/R/V insérée dans le modèle WVR à collisions, huit couches aux adresses fortement lexicales, puis huit couches contextuelles avec deux couches WVR restantes. Ce sont des ablations de chemin, non des contrôles factoriels isolés ; leur ordre motive des hypothèses sans en identifier les causes.

3 Protocole expérimental

3.1 Contrôles et chemin architectural

L'étude compare GQA, des variantes associatives à état fixe, des hybrides WVR-attention et le modèle contextuel W/R/V final à dix couches, tous autour de 98,2 millions de paramètres. Le chemin archivé évalue une couche W/R/V, huit couches fortement lexicales, huit couches contextuelles avec deux couches WVR, puis le modèle final à dix couches contextuelles avec normalisation RMS par tête.

L'entraînement historique séparé du modèle convolutionnel à micro-experts lexicaux sur 1 000 013 824 tokens est rapporté à part. Il ne constitue pas un contrôle apparié pour le tableau WVR à 100M tokens.

3.2 Données, tokenizer et budget

Les modèles utilisent un tokenizer de 200 019 entrées enregistré comme `o200k_base` et FineWeb-Edu sample-10BT. Son checksum n'a pas été archivé. Les runs principaux utilisent une séquence de 2 048, la seed 42, BF16 et 100 007 936 tokens. Chaque estimation finale moyenne 8 batches, soit 262 144 tokens, sans estimation d'incertitude. Le tableau 1 rapporte la dernière évaluation finie.

Les métriques GQA archivées utilisent 3×10^{-4} ; les runs récents utilisent 4×10^{-4} . Aucune configuration GQA réalisée n'est colocalisée avec son CSV ; dimensions, budget exact, tokenizer, données, seed et kernels ne sont donc pas vérifiables depuis l'artefact. La comparaison reste descriptive.

3.3 Mesures et règle de preuve

Nous rapportons l'entropie croisée next-token sur les données tenues à part, le nombre réel de paramètres, le nombre exact de tokens et le débit d'entraînement journalisé à l'étape d'évaluation finie. Qualité et débit constituent des affirmations séparées. Chaque entrée numérique d'un tableau doit correspondre à un artefact CSV ou JSON archivé ; les figures générées sont dérivées de ces fichiers et ne constituent pas la preuve primaire. Les proxies courts servent uniquement de filtres d'échec, plusieurs classements favorables sur proxy ne s'étant pas transférés au budget complet.

3.4 Diagnostics de contexte

Le rappel associatif, l'induction et l'apprentissage synthétique en contexte sont évalués selon la distance. Ils testent une récupération sélective des occurrences que la loss moyenne peut masquer. Les checkpoints sans attention terminés obtiennent une précision nulle sur les sondes archivées de rappel et d'induction à toutes les distances testées, tandis que GQA fournit le contrôle positif. Les revendications d'état persistant au-delà du champ réceptif convolutionnel restent donc non étayées et exigent des tests dédiés à 4-8k de contexte.

3.5 Matériel et vérifications numériques

Les runs complets utilisent un NVIDIA H100 80GB HBM3. Les tests FP32 couvrent la causalité et l'équivalence full/incrémental de W/R/V, ainsi que l'état fixe des primitives convolutionnelles et WVR. Les profils BF16 archivés ne soutiennent pas une équivalence exacte à 10^{-5} au niveau checkpoint. Le run final n'a pas de checkpoint ; les portes apprises sont irrécupérables. Les champs génériques top- k /Zipf du namespace CLI sont inactifs pour le module micro-expert réalisé, qui utilise le modulo défini en Section 2.

4 Résultats

4.1 Chemin architectural terminé à 100M tokens

WVR à collisions normalisées est la meilleure variante à état fixe. La condition à une couche W/R/V diffère de 0,003401. La condition huit couches fortement lexicale atteint 4,973798 et l'hybride contextuel 4,852761. La condition finale retire conjointement les deux couches WVR et ajoute RMSNorm par tête, atteignant

TABLE 1 – Résultats nominaux à 100M tokens lus depuis les CSV/JSON archivés. Le débit est un point journalisé à la dernière évaluation finie sur un H100. GQA n’a pas de configuration réalisée archivée et utilise 3×10^{-4} ; les runs récents utilisent 4×10^{-4} .

Mécanisme	Loss tenue à part ↓	Tokens/s ↑
GQA dense (QKV)	4,820476	111 355
Contexte associatif additif partagé	4,921405	124 363
WVR Stable Delta	4,898880	121 971
WVR Delta multi-échelles	4,905554	120 496
WVR à collisions normalisées	4,895427	122 282
WVR collision + 1 couche W/R/V lexicale	4,892026	123 145
WVR à valeur lexicale	4,963167	123 113
WVR Forge lexicale V3	4,900726	121 697
Forge V3 + normalisation des collisions	4,897241	121 312
Adresse compacte contexte + position	4,899072	122 764
W/R/V fortement lexical, 8/10 couches	4,973798	118 546
W/R/V contextuel, 8/10 couches	4,852761	118 936
W/R/V contextuel + RMSNorm, 10/10	4,706118	118 416

4,706118 à 118 416 tokens/s. Ces comparaisons n’isolent ni résidu lexical, ni normalisation, ni remplacement de couches.

La dernière ligne est inférieure de 0,114358 en loss et son point de débit est supérieur de 6,34% au point GQA. Ce n’est ni une estimation de supériorité architecturale ni un speedup reproductible : LR et provenance différent. Un GQA entièrement archivé à 4×10^{-4} est requis.

La forge lexicale réutilise la table d’objets lexicaux de rang 16 au moyen d’un réseau $16 \rightarrow 4 \rightarrow 128$. Son fort gain sur proxy court ne s’est pas transféré : Forge V3 atteint 4,900726 au budget complet, et sa combinaison avec la normalisation des collisions atteint 4,897241 sans améliorer la variante collision seule. Une valeur purement lexicale est nettement moins bonne à 4,963167, ce qui soutient le maintien de $\mathbf{v}_t = \mathbf{W}_v \mathbf{h}_t$. Les forges de lecture/écriture indépendantes ou faiblement couplées ont été rejetées sur proxies appariés ; leurs JSON bruts sont conservés comme ablations négatives plutôt que promus dans l’implémentation canonique.

4.2 Fiabilité des proxies

Les proxies courts appariés ont permis de détecter les échecs catastrophiques, mais n’ont pas correctement classé les losses finales à 100M tokens. Delta multi-échelles, Forge lexicale V3 et Forge V3 avec normalisation des collisions semblaient favorables sur proxy court et n’ont pas amélioré leur contrôle au budget complet. Aucune revendication du tableau 1 ne repose donc sur un proxy. L’adresse d’occurrence compacte contexte plus position échoue elle aussi à améliorer la normalisation des collisions au budget complet (4,899072 contre 4,895427).

4.3 Entraînement convolutionnel à budget plus long

L’entraînement historique séparé à un milliard de tokens du modèle convolutionnel sans attention compte 98 194 244 paramètres et atteint une loss tenue à part de 4,180625 à l’étape 30 000, avec un débit mesuré de 113 621 tokens/s. Son budget étant dix fois supérieur, il n’est pas utilisé pour classer les mécanismes QKV et WVR entraînés sur 100M tokens.

4.4 Équivalence incrémentale et état fixe

Les tests unitaires FP32 reproduisent les sorties de séquence complète sous exécution incrémentale token par token avec une tolérance de 10^{-5} pour les primitives convolutionnelles et WVR. Les profils BF16 archivés des checkpoints montrent des différences numériques plus grandes (environ 0,02–0,17 d’erreur absolue maximale) ; nous ne revendiquons donc pas une équivalence BF16 exacte au niveau checkpoint. Pour WVR, l’état récurrent

comprend une matrice associative de rang 128, l’adresse d’écriture précédente et, pour la normalisation des collisions, un vecteur diagonal de charge de rang 128. L’ablation d’adresse d’occurrence ajoute un unique compteur scalaire de position absolue. Ces formes d’état ne croissent pas avec la longueur du contexte généré.

Nous auditons le checkpoint convolutionnel exporté réellement utilisé plutôt que de déduire l’absence d’attention de sa configuration. Parmi 182 tenseurs, l’audit des noms et formes ne trouve aucune projection de requête, clé, valeur ou QKV fusionnée. Les chemins de compatibilité comme `self_attn` ne correspondent pas à une opération d’attention. Cet audit structurel ne remplace pas les tests d’exécution.

Les sondes archivées de rappel associatif et d’induction rapportent une précision nulle pour les checkpoints sans attention évalués. L’artefact synthétique GQA disponible ne documente pas un protocole identique ; aucune comparaison positive quantitative n’est donc revendiquée.

5 Travaux connexes

Les Transformers utilisent une compétition contextuelle query-key et une agrégation de valeurs (Vaswani et al., 2017). GQA partage les têtes key/value entre groupes de queries afin de réduire mémoire et bande passante (Ainslie et al., 2023). L’opération centrale W/R/V appartient à cette famille : les lectures jouent le rôle des queries, les écritures partagées celui des keys, et les valeurs restent contextuelles. Nous ne prétendons donc pas qu’un changement de vocabulaire crée un nouvel opérateur d’attention. La distinction étudiée est un résidu lexical lié, initialisé à zéro sur les écritures contextuelles, intégré aux résidus lexicaux feed-forward.

Le modèle final emploie aussi des composants établis : RoPE (Su et al., 2021), une normalisation par tête de R/W proche de query-key normalization (Henry et al., 2020), et des kernels d’attention exacte sensibles aux entrées/sorties (Dao et al., 2022). La question empirique porte sur leur combinaison avec les résidus lexicaux liés sous un budget de petit modèle.

RWKV, RetNet et Mamba développent des alternatives récurrentes ou à espace d’état au traitement linéaire de séquence (Peng et al., 2023; Sun et al., 2023; Gu & Dao, 2023). Nos ablations WVR à état fixe leur sont plus proches par motivation que le modèle W/R/V final, mais utilisent une matrice associative unique distincte et sont moins performantes dans nos contrôles. Le modèle final abandonne l’état fixe et conserve des caches W/V groupés ; il ne doit pas être présenté comme concurrent à état linéaire de ces méthodes.

6 Limites et impact élargi

L’étude actuelle se limite à environ 100 millions de paramètres, un tokenizer, un corpus web et une seed pour les runs principaux. Un résultat favorable n’établit ni le comportement à grande échelle ni une supériorité universelle sur GQA. Le débit dépend de l’implémentation et ne doit pas être généralisé à partir d’un seul H100.

L’étude n’établit pas le débit d’inférence en production. W/R/V conserve un cache W/V qui croît avec le contexte, comme un cache KV groupé. Prefill long contexte, temps jusqu’au premier token, latence de service et replay CUDA Graph restent hors des revendications.

Le conditionnement sur l’identité des tokens peut encoder des biais propres au corpus. FineWeb-Edu hérite des limites des grands corpus web, notamment une provenance incertaine, des déséquilibres de représentation et des contenus potentiellement nocifs. L’architecture ne supprime pas ces risques, et les gains d’efficacité peuvent réduire le coût de déploiement de modèles nuisibles.

WVR à état fixe ne conserve pas d’occurrences individuellement adressables. Ces contrôles négatifs motivent W/R/V contextuel ; ils ne décrivent pas le modèle final.

La comparaison principale n’est pas appariée en optimisation : GQA utilise 3×10^{-4} et W/R/V final 4×10^{-4} . Aucun checkpoint final W/R/V n’a été sauvegardé, empêchant l’inspection des portes lexicales. Les preuves actuelles ne séparent donc pas learning rate, RMSNorm, détails d’implémentation et résidu lexical. Il faut un GQA à 4×10^{-4} et un W/R/V au résidu lexical maintenu désactivé, avec checkpoints et idéalement plusieurs seeds. Les proxies courts doivent rester de simples filtres d’échec.

7 Conclusion

Nous avons évalué une configuration W/R/V complète avec têtes groupées, RMSNorm par tête, RoPE, SDPA, objets lexicaux liés et micro-experts. Le run de 98 195 224 paramètres atteint 4,706118 en évaluation et un point journalisé de 118 416 tokens/s. Il est numériquement inférieur au GQA archivé à 4,820476, tandis que WVR collision atteint 4,895427.

Cet ordre n'est pas une supériorité contrôlée : GQA emploie un autre LR et manque de configuration réalisée archivée. Aucun checkpoint W/R/V final n'existe. Il faut un GQA entièrement archivé, un contrôle sans résidu lexical, des contrôles factoriels couches/normalisation, des checkpoints et des mesures répétées de débit.

Références

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa : Training generalized multi-query transformer models from multi-head checkpoints. 2023.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention : Fast and memory-efficient exact attention with io-awareness. 2022.
- Albert Gu and Tri Dao. Mamba : Linear-time sequence modeling with selective state spaces. 2023.
- Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. 2020.
- Bo Peng et al. RwkV : Reinventing runs for the transformer era. 2023.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer : Enhanced transformer with rotary position embedding. 2021.
- Yutao Sun et al. Retentive network : A successor to transformer for large language models. 2023.
- Ashish Vaswani et al. Attention is all you need. 2017.

A Liste de contrôle pour la reproductibilité

L'instantané du dépôt accompagnant cette révision contient :

- le code du modèle et les configurations JSON réalisées des conditions WVR et W/R/V récentes ; la ligne GQA possède des métriques mais pas de configuration réalisée archivée ;
- le nom du tokenizer, la taille du vocabulaire, l'identifiant du dataset, le ratio déterministe d'évaluation, les seeds, longueurs de séquence, taux d'apprentissage et budgets exacts de tokens ;
- les métriques CSV par étape ou mesures JSON archivées pour chaque ligne du tableau, y compris les ablations négatives ;
- les scripts déterministes de collecte et de tracé des diagnostics de checkpoint et de contexte ;
- les tests de causalité W/R/V, formes des têtes groupées, identité de table lexicale liée, initialisation lexicale neutre, RoPE et équivalence FP32 full/incrémental, ainsi que les tests WVR à état fixe ;
- un audit des noms et formes du checkpoint historique sans attention, explicitement non appliqué au modèle final avec attention.

Les checkpoints lourds, l'historique Git, les credentials, adresses cloud, noms d'hôtes, chemins utilisateur locaux et logs transitoires sont exclus. Aucun checkpoint final W/R/V n'existe et le run n'enregistre aucun commit Git. La source relue corrige après le run les masques causaux, le cache par chunks et l'initialisation de projection résiduelle : c'est une implémentation corrigée par audit, non le snapshot historique octet pour octet, et le résultat doit être répliqué. Les chemins identifiants sont remplacés par [REDACTED] ; le supplément double aveugle doit être une archive scannée sans .git.